

Spooky Box Design Philosophy, Functional Requirements, and Software Architecture

1. Product Vision

Spooky Box is an experimental field instrument that combines radio scanning, microphone capture, EMF sensing, visual feedback, session recording, and performable interaction into a cohesive handheld device.

The device should feel like a real instrument rather than a novelty effect. Its behavior should be explainable in engineering terms, but its presentation should leave room for ambiguity, ritual, and emotional interpretation. The experience should live somewhere between field recorder, radio scanner, synthesizer controller, and haunted measuring device.

The system should prioritize immediacy, responsiveness, and legibility. Controls should feel physical and intentional. Visual feedback should reinforce what the device is doing without requiring the user to read a detailed screen constantly.

2. Core Design Philosophy

2.1 Instrument First

Spooky Box should behave like an instrument with modes, ranges, sensitivity, modulation, and feedback. It should not feel like a menu-driven toy where effects are simply triggered. The user should be able to build intuition over time: how the knobs respond, how the scan engines behave, how EMF affects the system, and how radio activity changes the visual texture.

2.2 Stable Meaning, Variable Texture

The system should keep core meanings consistent across modes:

- EMF level represents field disturbance relative to a calibrated ambient baseline.
- Radio activity represents detected activity or change in the incoming radio audio.
- Scan position represents where the radio engine is moving through its current territory.
- Session state represents whether the device is idle, armed, recording, or reviewing.

Different scan engines and modes may present these meanings with different visual textures, but the underlying interpretation should remain stable.

For example, the LED matrix may use different animations for classic scan, orbit scan, or interband scan, but EMF should still generally influence color, brightness, speed, or intensity, while radio activity should still inject liveliness, flicker, shimmer, or transient motion.

2.3 Semantic UI Over Raw Pixels

The main application should describe what is happening, not micromanage every visual output.

Instead of pushing pixels to the LED matrix from the main application loop, the system should eventually send semantic state such as:

- current mode
- scan engine
- EMF level
- EMF bucket
- radio activity
- scan position
- band
- recording state
- one-shot events such as signal spikes or band transitions

The UI layer should then decide how to render that state.

2.4 Prototype Quickly, Architect for Separation

The Teensy-only prototype is allowed to run everything locally. However, the codebase should be structured as if a future UI coprocessor exists. This allows the project to move forward without blocking on multi-controller hardware while reducing future rewrite risk.

The recommended path is:

1. Implement and validate behavior on the Teensy-only prototype.
2. Introduce clean software boundaries.
3. Build a UI subsystem PCB with a coprocessor.
4. Move animation, input scanning, and UI-side timing to the coprocessor.
5. Preserve the Teensy as the source of truth for device behavior.

3. Hardware Architecture Direction

3.1 Main Controller

The Teensy remains the primary controller for core device behavior.

The Teensy owns:

- radio control
- audio capture and processing
- microphone and radio recording
- session state
- storage
- scan engines

- EMF sensor processing
- high-level mode state
- mapping input events to behavior
- deciding what state should be sent to the UI layer

The Teensy should not eventually be responsible for frame-by-frame LED matrix animation timing.

3.2 UI Coprocessor

A UI coprocessor is recommended for the final architecture and should be planned for now. The first full prototype does not need to depend on it, but the software should make space for it.

The UI coprocessor owns:

- button scanning and debounce
- encoder quadrature decoding
- button LED PWM, blinking, and breathing effects
- 9x9 RGB LED matrix animation rendering
- LED matrix driver communication
- UI-side brightness, gamma, and color correction
- animation timing independent of Teensy loop jitter
- possibly TFT/OLED rendering in a future revision
- coprocessor health/status reporting

The recommended default MCU for the UI coprocessor is RP2040 because it provides ample performance, dual cores, flexible I/O, strong ecosystem support, PIO for future timing-sensitive tasks, and a practical bootloader/update story.

3.3 Display Ownership

Near-term, the Teensy may continue to draw the TFT or OLED display. This keeps the first coprocessor integration focused on LED matrix animation, input scanning, and button LED feedback.

Long-term, it may be advantageous for the UI coprocessor to own all visual rendering, including the TFT/OLED, especially if the display becomes animation-heavy or physically belongs on the UI board.

The architecture should not assume forever that the Teensy owns the display.

3.4 LED Matrix Driver

The prototype uses an Adafruit IS31FL3741 13x9 PWM RGB LED matrix breakout. The final design is expected to use a square matrix, likely 9x9.

During prototyping, the 13x9 breakout can be treated as a physical surface with a centered 9x9 logical viewport. This allows the animation design to target the intended final form factor while using available hardware.

The IS31FL3741-style architecture is appropriate for Spooky Box because the LED matrix is an expressive instrument display, not a high-frame-rate video display. The matrix should show pulses, gradients, scan movement, field intensity, radio texture, signal events, and mode identity.

The driver code should remain modular so the final PCB can use:

- IS31FL3741 or similar matrix driver
- another ISSI/Lumissil LED driver
- smart RGB LEDs
- a UI coprocessor-controlled display surface

Application logic should not depend directly on IS31FL3741 register details.

4. LED Matrix Role and Behavior

4.1 Primary Role in Field Mode

In field mode, the LED matrix should primarily indicate EMF level through some combination of:

- color
- brightness
- animation speed
- visual intensity
- density
- instability

The matrix should make the device feel reactive to the surrounding field.

4.2 Radio Activity as Texture

Radio audio activity should influence the animation as texture rather than replacing the core EMF meaning.

EMF determines the weather. Radio activity makes the ghosts move.

Radio activity can modulate:

- flicker
- shimmer
- spark amount
- pulse depth
- jitter
- scan trail length
- asymmetry
- transient flashes
- button LED accents

The radio activity metric should be derived from audio processing and may include:

- RMS level
- spectral flux
- transient hits
- squelch/open-state cues
- signal confidence or activity heuristics

4.3 Per-Scan-Engine Animation Identity

Each scan engine should have its own matrix animation style, while still preserving the same core meanings.

Examples:

- Classic scan: clean sweep line, tuning cursor, scan trail, or rhythmic pulse.
- Orbit scan: particles orbiting attractors or converging around a center.
- Random/chaotic scan: scattered sparks, unstable flashes, noisy texture.
- Interband scan: color-domain shifts or wipes when moving between bands.
- Calibration: settling particles, stabilizing center, or convergence animation.
- Instrument mode: modulation-focused patterns that respond directly to the magnetometer as a control source.

The display can explain the mode explicitly, while the LED matrix gives the mode a felt signature.

5. Radio and Scan Architecture

5.1 Scan Mode vs Territory

The scan engine should separate how movement happens from where movement is allowed to happen.

Scan mode describes motion:

- off/manual
- linear up
- linear down
- ping-pong
- random
- seek-like behavior
- orbit or other future engines

Band or territory describes the available frequency space:

- FM
- MW/AM
- SW
- LW

- weather, if supported
- curated interband territory
- custom territories

The scan engine should operate over TuneTargets or ScanRanges, not raw global frequencies.

5.2 Tune Target Model

The radio layer should receive a structured target such as:

- band/mode
- frequency
- step size
- optional range metadata
- RF input path

The scan engine should not directly call low-level radio library functions. It should emit a desired target. The radio controller decides whether that target is a cheap same-band tune or a more expensive mode/band transition.

5.3 Band Switching and Interband Scanning

Band switching is not the same as tuning.

Changing frequency within FM is relatively cheap. Moving between FM and AM/SW/LW may require mode changes, property updates, antenna routing changes, muting, and settling time.

Interband scanning should be modeled as movement through a curated list of ranges rather than one continuous frequency axis.

A scan territory may contain entries such as:

- FM 87.5 to 108.0 MHz
- AM 520 to 1710 kHz
- SW 5.8 to 6.2 MHz
- SW 9.4 to 10.0 MHz

The scanner may walk through these ranges linearly or choose among them randomly with weights.

5.4 Si4732 / Si4735 Library Considerations

The current library treats AM, MW, LW, SW, and SSB-like behavior under AM-mode power-up, while FM is a separate power-up mode. Therefore, software should model:

- FM as FM mode
- MW/AM as AM mode
- SW as AM mode
- LW as AM mode

The user's experimentation suggests that SW should be treated as an AMI-path mode, not as a direct FMI-path mode. Although simplified diagrams sometimes label an FM/SW antenna path, practical testing showed SW tuning had meaningful noise changes when the antenna was connected to AMI rather than FMI.

Final hardware should therefore plan for:

- FM using FMI
- AM/MW using AMI with an AM-style antenna network
- SW using AMI with a whip, long-wire, or wideband antenna network
- possible switching or configurable routing into AMI

AM and SW both wanting AMI is a key hardware design constraint because MW and SW may prefer different antenna networks.

5.5 Audio Behavior During Band Changes

Band changes may produce pops, mutes, or discontinuities. The system should make this intentional.

Recommended behavior:

- emit a BandTransitionStart event
- fade or mute radio audio briefly
- switch RF path and radio mode
- tune to the target frequency
- wait for tune/status completion or a short settle period
- emit BandTransitionComplete
- restore audio with a short fade

The LED matrix and button LEDs can use these events to make transitions feel deliberate.

6. Classic Mode Control Philosophy

6.1 Current Control Direction

Classic mode currently uses:

- Encoder 0 turn: manual tuning in off mode; jump rate in linear scan modes
- Encoder 0 click: change scan mode
- Encoder 1 turn: jump size in linear scan modes; tuning step size in off/manual mode
- Encoder 1 click: band or territory switching

This creates two conceptual axes:

- Encoder 0: motion
- Encoder 1: scale and territory

6.2 Manual Mode Analog Feel

Although the Si4732 is a DSP tuner, manual classic tuning should feel analog-ish. The tuning experience should feel like moving through a medium rather than setting a number.

Useful techniques:

- tuning inertia
- smoothing between target frequency and current displayed/commanded frequency
- micro-drift in display or modulation values
- station capture or sticky frequency behavior using RSSI/SNR/audio activity
- nonlinear encoder response based on turn velocity
- friction zones near signals
- subtle imperfect stepping
- speed-dependent noise or visual instability

Manual mode should feel different from scan mode:

- Manual: continuous, tactile, weighted, expressive.
- Scan: rhythmic, mechanical, clocked, procedural.

7. EMF Sensor Calibration and Processing

7.1 Purpose

The magnetometer is used primarily as a relative disturbance sensor, not as an absolute compass. It should measure deviation from ambient magnetic conditions.

7.2 Startup Calibration

On boot, the device should collect magnetometer samples for a fixed window, such as 2 to 5 seconds.

For each sample:

- compute magnitude: $\sqrt{xx + yy + z*z}$

The startup calibration should reject outliers or spikes using a simple robust method such as median filtering or ignoring values beyond a standard deviation threshold.

It should compute:

- baselineMagnitude
- noiseFloor

The noise floor should have a reasonable minimum to avoid divide-by-zero and hypersensitivity.

7.3 Continuous EMF Calculation

For each new sample:

- compute magnitude
- compute $\Delta = \text{abs}(\text{magnitude} - \text{baselineMagnitude})$
- normalize Δ from $[\text{noiseFloor}, \text{threshold}]$ to $[0.0, 1.0]$
- clamp to $[0.0, 1.0]$

Then apply exponential shaping:

- $\text{shaped} = \text{pow}(\text{normalized}, \text{gamma})$
- gamma should be tunable, with a default around 2.2 and a useful range around 2.0 to 3.0

Then apply smoothing to produce a stable output.

The system should expose:

- raw magnitude
- normalized EMF level
- shaped EMF level
- smoothed EMF level
- discrete EMF bucket

7.4 Discrete Buckets

The shaped EMF value can be grouped into N discrete buckets:

- $\text{bucket} = \text{floor}(\text{shaped} * N)$
- clamp to $[0, N - 1]$

Because shaping is exponential, higher buckets require stronger signals while lower ranges retain useful resolution.

7.5 Adaptive Baseline

Adaptive baseline drift should be configurable at runtime.

Enable adaptive baseline by default in field mode. Disable adaptive baseline by default in instrument mode.

When enabled:

- slowly adapt baselineMagnitude toward the current magnitude
- only adapt when the signal is quiet
- do not adapt during spikes or high activity
- use a very small α , such as 0.0005 to 0.005

When disabled:

- continue computing EMF relative to the existing baseline
- do not update baselineMagnitude or noiseFloor

This matters because in instrument mode the user may intentionally manipulate the magnetic field as a modulation source. Adaptive baseline drift would make the control feel like it is moving under the user's hand.

Manual recalibration should remain available regardless of adaptive mode.

8. Session Recording Requirements

8.1 Current Prototype State

The Teensy-only prototype currently supports field mode with a classic scan engine, session recording with radio and microphone audio, TFT display output, and LED matrix animations.

Recording worked acceptably when only the TFT was active, but adding LED matrix animation has created buffer stress during sessions.

This is an important architectural signal. The LED matrix and UI animation workload should be offloaded or isolated rather than endlessly optimized inside the main recording path.

8.2 Recording Priority

Audio recording reliability should take priority over UI animation smoothness.

The system should avoid blocking or jitter-inducing operations in the recording-critical path. Display updates, LED matrix updates, button LED effects, and input polling should not be allowed to starve audio buffers or storage writes.

8.3 Coprocessor Motivation

The UI coprocessor architecture exists partly to protect session recording from UI timing and bus traffic.

Offloading LED matrix rendering, input scanning, and button LED PWM should reduce pressure on the Teensy during active recording sessions.

9. Software Architecture

9.1 High-Level Split

The software should be organized around domain ownership.

Main application domain:

- modes
- scan engines
- radio control
- audio processing
- recording
- storage
- EMF processing
- settings
- session lifecycle
- mapping inputs to behavior

UI domain:

- display rendering
- LED matrix scenes
- button LED feedback
- input scanning/debouncing/decoding
- visual transitions
- UI-side animation timing

Communication domain:

- shared protocol
- state packets
- event packets
- input event packets
- health/status packets
- firmware update management placeholder

9.2 Suggested Codebase Layout

A useful project layout:

/spooky-box /common Protocol.h UiState.h UiEvents.h RadioTypes.h SensorTypes.h FixedPoint.h

/firmware-main-tensy App.cpp FieldMode.cpp InstrumentMode.cpp RadioController.cpp AudioEngine.cpp
SessionRecorder.cpp SensorManager.cpp EmfProcessor.cpp ScanEngines/ ClassicScanEngine.cpp
OrbitScanEngine.cpp UiLink.cpp LocalUiCoproprocessor.cpp

/firmware-ui-coproprocessor UiApp.cpp UiProtocol.cpp MatrixRenderrer.cpp DisplayRenderrer.cpp
ButtonLedRenderrer.cpp InputScanner.cpp scenes/ ClassicScanScene.cpp OrbitScene.cpp
CalibrationScene.cpp MenuScene.cpp InstrumentScene.cpp drivers/ IS31FL3741Driver.cpp DisplayDriver.cpp

/tools protocol-test/ asset-converter/

/docs protocol.md architecture.md

9.3 UI Coprocessor Abstraction

The application should depend on an interface, not a specific transport.

Example concept:

```
class UiCoprocessor { pollInputEvents(); sendState(UiState state); sendEvent(UiEvent event);  
sendCommand(UiCommand command); getStatus(); }
```

There should be at least two implementations over time:

- LocalUiCoprocessor: current Teensy-local implementation
- RemoteUiCoprocessor: future external coprocessor implementation

This lets the project prototype locally while preserving the eventual hardware boundary.

9.4 State Packets vs Event Packets

The system should separate persistent state from one-shot events.

Persistent UI state answers: what is true right now?

Examples:

- mode
- scene
- band
- scan engine
- recording state
- EMF level
- EMF bucket
- radio activity
- scan position
- scan velocity
- PTT state
- battery state

One-shot UI events answer: what just happened?

Examples:

- band transition started
- band transition completed
- signal spike
- session armed
- recording started
- recording stopped

- PTT down
- PTT up
- calibration started
- calibration completed
- error occurred

Commands answer: what should the UI side explicitly do?

Examples:

- set brightness
- set scene
- set palette
- set button LED override
- enable/disable animation
- enter diagnostic mode

9.5 Input Boundary

The UI coprocessor should report decoded but behavior-neutral input events.

For example:

- encoder 0 moved +3
- encoder 1 moved -1
- encoder 0 clicked
- PTT button down
- PTT button up

The Teensy decides what those inputs mean based on current mode.

The coprocessor should not decide that encoder 0 means tuning, scan rate, menu navigation, or parameter editing. It only reports the input.

9.6 Transport

The exact transport can be chosen later. UART or SPI are likely better choices than I2C for the Teensy-to-coprocessor link, especially if firmware update support is planned.

The architecture should support:

- regular state updates from Teensy to coprocessor
- asynchronous input events from coprocessor to Teensy
- one-shot events from Teensy to coprocessor
- health/status queries
- future firmware update mode

The protocol should include versioning and enough validation to detect malformed packets.

9.7 Firmware Update Management

The Teensy should eventually be able to manage coprocessor firmware updates, but this should not be implemented until the coprocessor MCU, bootloader, transport, and image format are chosen.

The architecture should reserve a lifecycle manager with states such as:

- not present
- running
- bootloader
- updating
- verifying
- fault

Hardware should plan for Teensy control of:

- coprocessor reset
- coprocessor boot/BOOTSEL pin
- communication transport
- debug/programming access

A future update flow may be:

1. Teensy queries coprocessor firmware version.
2. Teensy compares against expected bundled version.
3. If needed, Teensy places coprocessor into bootloader mode.
4. Teensy streams firmware image in chunks.
5. Coprocessor verifies checksum or signature.
6. Coprocessor reboots.
7. Teensy confirms version and resumes normal operation.

For now, only placeholders and clean interfaces are required.

10. PCB Prototyping Strategy

10.1 Rationale for Split Prototypes

The project should avoid putting every risky subsystem onto one first integrated PCB. Some parts are straightforward, while others are unpredictable and deserve isolated testing.

The tricky areas include:

- radio RF performance
- AM/SW antenna routing
- audio noise
- power supply noise
- LED matrix current and bus traffic

- display bus activity
- mechanical alignment
- firmware update wiring

Splitting the PCB prototypes allows progress on stable subsystems without being blocked by RF or power issues.

10.2 Recommended First PCB: UI Subsystem

Given that the Teensy-only prototype is now experiencing buffer stress after adding LED matrix animations, the first PCB prototype should likely be a UI subsystem board.

The UI subsystem board should include:

- RP2040 or chosen UI coprocessor
- LED matrix driver or connector
- 9x9 RGB LED matrix or breakout-compatible footprint
- button LED drivers/PWM capability
- button and encoder input hardware
- TFT/OLED connector, even if the Teensy initially still drives display
- Teensy communication link
- coprocessor reset and boot pins controlled by Teensy
- debug/programming headers
- test pads on all important buses
- current measurement options
- power enable or isolation options

The first goal is to prove:

- the Teensy can record reliably while the UI coprocessor handles LED animation and input
- the state/event protocol is workable
- LED matrix animations are smooth
- button LEDs are responsive
- input events are decoded reliably
- firmware/update wiring assumptions are valid, even if full updater software is not complete

10.3 Radio/RF Prototype Board

The radio/RF subsystem should likely remain separate early because antenna behavior is harder to predict.

A radio test board should include:

- Si4732/Si4735 radio circuit
- AMI and FMI access
- configurable antenna networks
- jumpers or 0-ohm links for routing experiments
- RF test points
- audio output test points

- options for ferrite, whip, long-wire, or external antenna input
- footprints for optional filters, coupling components, and matching networks

This board should be designed for experimentation and bodge-friendly iteration.

10.4 Integration PCB

After UI and RF lessons are learned, a larger integration PCB can combine:

- Teensy or main MCU section
- audio capture and routing
- radio subsystem
- UI coprocessor subsystem
- display and LED matrix
- power and battery management
- storage
- controls
- enclosure-driven mechanical layout

The first integrated PCB should still be considered a validation board, not a final production board.

11. Practical Engineering Principles

11.1 Protect Audio and Recording

Avoid letting visual feedback interfere with audio buffer timing or recording reliability.

UI updates should be scheduled, offloaded, or deprioritized when necessary.

11.2 Make Transitions Intentional

Band changes, scan direction changes, recording start/stop, EMF calibration, and signal spikes should generate explicit events. These events can be used by the display, matrix, and button LEDs to create a coherent device personality.

11.3 Keep Hardware Abstractions Honest

The software should model real hardware constraints explicitly:

- band switching cost
- RF input path
- AMI vs FMI differences
- per-band step sizes
- antenna configuration
- tune settling
- audio muting windows
- UI transport health

- coprocessor availability

Hiding these realities too deeply will make later bugs harder to diagnose.

11.4 Prefer Semantic State Over Raw Control

The main application should not need to know how to draw a classic scan shimmer or an orbit pulse. It should report the state of the device. The UI layer should render that state.

11.5 Design for Debugging

Prototype PCBs should include:

- test pads
- labeled headers
- reset/boot access
- serial debug
- measurement jumpers
- optional footprints
- clear silkscreen
- isolation points for noisy subsystems

The first board should be easy to probe, bodge, and understand.

12. Near-Term Next Steps

1. Keep the Teensy prototype as the behavioral reference.
2. Introduce or formalize the UiCoprocesor interface in the codebase.
3. Implement a LocalUiCoprocesor backend that preserves current behavior.
4. Define UiState, UiEvent, UiCommand, and InputEvent structures.
5. Stop allowing core application code to directly depend on LED matrix implementation details.
6. Begin UI subsystem PCB design around RP2040 or chosen coprocessor.
7. Include reset/boot/update wiring from the Teensy to the coprocessor.
8. Plan for the UI board to handle LED matrix animation, button LEDs, and input scanning first.
9. Leave TFT/OLED ownership flexible.
10. Continue radio/RF experimentation separately, especially around SW behavior through AMI and antenna routing.

13. Summary

Spooky Box should be built as a responsive, expressive field instrument. The Teensy should remain responsible for the core behavior: radio, audio, sessions, sensors, scan engines, and state. The UI layer should eventually become an intelligent presentation subsystem, likely built around an RP2040 coprocessor that owns LED matrix animation, input scanning, button LEDs, and possibly display rendering.

The project has reached the point where a split PCB prototyping strategy makes sense. The Teensy-only prototype has validated the core experience but is beginning to show buffer stress when recording,

displaying, and animating simultaneously. That is exactly the kind of stress the coprocessor architecture is meant to solve.

The best next hardware step is likely a focused UI subsystem PCB, not a full integrated product PCB. This keeps development moving while proving the architecture that will let Spooky Box grow into a reliable, polished embedded instrument.